

实验报告

分治与递归算法

截止提交日期：2025 年 5 月 30 日

学号：_袁责铭_ 姓名：_2024012456_

一、设计目的

- 理解分治算法的基本思想，掌握将复杂问题分解为若干规模较小子问题的方法。
- 掌握递归算法的设计过程，能够正确确定递归关系和递归终止条件。
- 通过具体问题的程序实现，提高算法分析、代码编写和实验验证能力。

二、设计内容

二叉搜索树具有如下性质：对于任意一个结点，其左子树中所有结点的值都小于该结点，右子树中所有结点的值都大于该结点。给定 n 个不同的关键字，求可以构成多少种不同的二叉搜索树。

例如：当 $n = 1$ 时，只有 1 种；当 $n = 2$ 时，有 2 种；当 $n = 3$ 时，有 5 种。请使用分治与递归思想设计算法，计算 n 个不同关键字能够构成的不同二叉搜索树数量。

三、算法设计方案

3.1 如何选择根结点

设 n 个不同关键字从小到大排列为 $\{k_1, k_2, \dots, k_n\}$ 。依次枚举每个关键字 k_i ($i = 1, 2, \dots, n$) 作为当前子树的根节点。由二叉搜索树的性质可知，选定 k_i 为根后， $\{k_1, \dots, k_{i-1}\}$ 共 $i-1$ 个关键字必须位于左子树， $\{k_{i+1}, \dots, k_n\}$ 共

$n-i$ 个关键字必须位于右子树。

3.2 如何划分左子树和右子树

由于二叉搜索树的结构只取决于关键字的相对大小关系而非具体值，因此：

- ◆ 左子树：由 $i-1$ 个不同关键字构成，其可能的结构数等价于 $f(i-1)$ 。
- ◆ 右子树：由 $n-i$ 个不同关键字构成，其可能的结构数等价于 $f(n-i)$ 。
- ◆ 对于固定的根 k_i ，左右子树相互独立，总方案数为 $f(i-1) \times f(n-i)$ 。

3.3 递归关系

对所有可能的根节点枚举求和，得到递归关系：

$$f(n) = \sum f(i-1) \times f(n-i), i = 1, 2, \dots, n$$

此即经典的卡特兰数（Catalan Number）递推公式，其封闭形式为： $C(n) = C(2n, n) / (n+1)$ 。

3.4 递归终止条件

当 $n = 0$ 时，表示空树，结构唯一， $f(0) = 1$ 。

当 $n = 1$ 时，只有单节点树，结构唯一， $f(1) = 1$ 。

这两种情况作为递归的边界条件，保证递归能够正确终止。

四、算法适用性分析

本问题高度适合分治算法，具体理由如下：

- ◆ 可分解性： n 个关键字的 BST 计数问题，可通过枚举根节点分解为两个规模更小的同类子问题（左子树和右子树各自的 BST 计数）。
- ◆ 子问题同构：子问题与原问题结构完全相同，均为「给定 k 个关键字，求 BST 数量」，满足递归求解的基本条件。

- ◆ 结果可合并：左子树有 $f(i-1)$ 种结构，右子树有 $f(n-i)$ 种结构，两者相互独立，组合数为乘积，最终对所有根节点的枚举求和即可得到原问题答案。
- ◆ 明确的递归边界： $f(0) = f(1) = 1$ 构成清晰的递归终止条件，保证算法不会无限递归。

五、核心代码实现

使用 Python 语言实现，采用递归结合记忆化（Memoization）的方式，在保留递归结构清晰的同时避免重复计算。

► 代码如下：

```
# 二叉搜索树计数（递归 + 记忆化）

# 使用字典缓存已计算结果，避免重复递归
memo = {}

def count_bst(n):
    """
    计算 n 个不同关键字能构成的不同二叉搜索树数量。
    利用分治思想：依次枚举每个关键字作为根节点，
    左子树关键字数为 i-1，右子树关键字数为 n-i。
    """
    # 递归终止条件：0 个或 1 个关键字只有 1 种结构
    if n <= 1:
        return 1

    # 若已计算过，直接返回缓存结果
    if n in memo:
        return memo[n]

    total = 0
    # 枚举根节点：第 i 个关键字作为根（i 从 1 到 n）
    for i in range(1, n + 1):
        left = count_bst(i - 1)    # 左子树种数
        right = count_bst(n - i)   # 右子树种数
        total += left * right      # 左右组合相乘

    memo[n] = total    # 缓存本次结果
```

```
return total

if __name__ == '__main__':
    n = int(input('请输入关键字数量 n: '))
    print(f'n = {n} 时，不同二叉搜索树的数量为: {count_bst(n)}')
```

代码说明：memo 字典用于缓存已计算的结果，使每个子问题只被计算一次，将朴素递归的指数级时间复杂度降低为 $O(n^2)$ 。

六、测试与结果分析

6.1 测试结果

选取 8 组测试数据，涵盖边界情况、小规模及中等规模输入，测试结果如下：

输入 n	输出结果（二叉搜索树数量）
1	1
2	2
3	5
4	14
5	42
6	132
10	16796

6.2 结果分析

测试结果完全符合预期，具体分析如下：

- ◆ $n = 1$ 时， $f(1) = 1$ ，仅有单节点树，结果正确。
- ◆ $n = 2$ 时， $f(2) = f(0) \times f(1) + f(1) \times f(0) = 1 + 1 = 2$ ，对应以关键字 1 为根和以关键字 2 为根两种形态，结果正确。

- ◆ $n = 3$ 时, $f(3) = f(0) \times f(2) + f(1) \times f(1) + f(2) \times f(0) = 2+1+2 = 5$, 与题目示例吻合。
- ◆ $n = 4$ 时, $f(4) = 14$; $n = 5$ 时, $f(5) = 42$ 。以上均为标准卡特兰数序列 $(1, 1, 2, 5, 14, 42, 132, \dots)$, 验证了算法的正确性。
- ◆ $n = 10$ 时, $f(10) = 16796$, 计算速度极快, 记忆化优化效果显著。

七、总结与讨论

7.1 对分治思想的理解

分治算法的核心在于「分而治之」——将规模较大的问题递归地分解为若干结构相同、规模更小的子问题, 分别求解后再合并结果。本实验中, n 个关键字的 BST 计数问题通过枚举根节点, 自然地分解为左右子树两个独立的子问题, 体现了分治的思想精髓。子问题之间相互独立、结果以乘积形式合并, 是分治算法适用的典型场景。

7.2 对递归算法设计的认识

设计递归算法需要清楚地明确两点: 其一是递归关系, 即如何用更小规模的问题来表达当前问题; 其二是递归终止条件, 即何时停止递归、直接返回结果。本实验中递归关系 $f(n) = \sum f(i-1) \times f(n-i)$ 清晰直接, 终止条件 $f(0) = f(1) = 1$ 简单明确, 使得代码逻辑易于理解和验证。

7.3 实验过程中遇到的问题及解决方法

在最初的纯递归实现中, 发现当 n 较大时 (如 $n = 20$) 计算极慢, 甚至出现明显卡顿。原因在于纯递归存在大量重复计算: 例如 $f(3)$ 在计算 $f(5)$ 时会被反复调用。为此引入了记忆化 (Memoization) 技术, 用字典缓存已计算的结果, 使每个子问题只被计算一次, 问题得到解决。

7.4 递归算法可能存在的效率问题

- ◆ 重复计算: 朴素递归中, 同一子问题可能被多次求解, 导致时间复杂度呈指数级增长 ($O(2^n)$)。

- ◆ 栈溢出：递归深度过大时，系统调用栈可能溢出（Stack Overflow），尤其在 n 极大时。
- ◆ 空间开销：递归调用栈本身需要占用内存，相比迭代实现开销更大。

7.5 改进思路

- ◆ 记忆化递归（Top-Down DP）：如本实验所实现，使用 memo 字典缓存子问题结果，时间复杂度降低为 $O(n^2)$ ，空间复杂度 $O(n)$ 。
- ◆ 动态规划（Bottom-Up DP）：从 $f(0)$ 、 $f(1)$ 开始，自底向上依次填表，完全消除递归调用栈的开销，更适合 n 较大的场景。
- ◆ 数学公式：卡特兰数有封闭公式 $C(n) = C(2n, n)/(n+1)$ ，可在 $O(n)$ 时间内精确计算，是最高效的方案，但需要高精度整数运算库支持（ n 较大时结果超出普通整型范围）。

综上，本实验通过二叉搜索树计数问题，加深了对分治思想与递归设计的理解，体会到记忆化对提升递归效率的重要性，也认识到从递归向动态规划过渡的自然逻辑。